TAMPRIX – METER DATA TAMPERING PROJECT

SUMMER INTERNSHIP – I REPORT

Submitted by

KRISHNASHREE D - 2023105539

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY: CHENNAI 600025

MAY – JUNE 2025

TAMPRIX – METER DATA TAMPERING PROJECT

SUMMER INTERNSHIP – I REPORT

Submitted by

KRISHNASHREE D - 2023105539

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY: CHENNAI 600025

MAY – JUNE 2025

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this report titled, **"TAMPRIX – METER DATA TAMPERING PROJECT,"** is the Bonafide work of **KRISHNASHREE D (2023105539)** for the 5th Semester, who carried out the project work for **Summer Internship-I,** in the months of **May - June 2025** under the guidance of **Mr. Prasad, Executive Engineer, TNEB**. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.M.A Bhagyaveni

HEAD OF THE DEPARTMENT

Professor

Department of ECE

College of Engineering Guindy

Anna University

Chennai-600025

ACKNOWLEDGEMENT

I express my sincere gratitude to the Dean, **Dr.K.S.Easwarakumar**, Professor, College of Engineering, Guindy for his support throughout the project.

I extend my heartfelt appreciation to my Head of the Department, **Dr.M.A Bhagyaveni,** Professor, Department of Electronics & Communication Engineering for her support throughout the project.

I am immensely grateful to my project supervisor and coordinator, **Mr. Prasad**, Executive Engineer, Tamil Nadu Generation and Distribution Corporation Limited, for his unwavering assistance, patience, valuable guidance, technical mentorship, and encouragement in my project.

Certificate

POWERING TAMILNADU'S PROGRESS...

TAMILNADU POWER DISTRIBUTION CORPORATION LIMITED

This is to certify that **Miss. Krishnashree D** has completed her **Internship Training** in the Office of Chief Engineer/Information Technology, TAMILNADU POWER DISTRIBUTION CORPORATION LIMITED **(TNPDCL)** for her **B.E (ECE)** at **College of Engineering, Guindy - Anna University, Chennai** during the period **27.05.2025 to 20.06.2025.**.

24/06/2025

EXECUTIVE ENGINEER /
SYSTEM/ADMIN/ IT.
EXECUTIVE ENGINEER
COMPUTER CENTRE
TANGEDCO, NPKRR MAALIGAI,
144, ANNA SALAI, CHENNAI-600 002.

Date: 20/06/2025

Place: Chennai.

ABSTRACT

Meter tampering has become a significant challenge for utility providers, resulting in energy theft, revenue loss, and operational inefficiencies. Traditional detection methods, which rely on manual inspections and isolated event logging, are reactive and limited in scale. This project aims to design a comprehensive, proactive system for tampering detection and prediction using simulated data, machine learning, and interactive visualization tools.

The system leverages SCILAB and Simulink to simulate tampering scenarios based on abnormal current drops and periodic manipulation. Detected tampering events are labelled and processed through predictive models such as ARIMA and Random Forest to forecast future tampering patterns. A user-friendly Streamlit dashboard displays real-time detection, risk forecasts. The complete solution is hosted online and secured via login-based access. This full-stack approach not only improves early detection but also supports operational planning and inspection prioritization for utilities.

Table of Contents

Chapter No.	Title	Page No.
1	Introduction	9
2	Software Requirements & Hardware Requirements	14
3	Data Acquisition and Infrastructure Setup	16
4	Tampering Behaviour Simulation Using Scilab And Simulink	18
5	Tampering Prediction and Forecasting	31
6	Login Page Implementation	35
7	Dashboard Page Implementation	43
8	Data Table and Analysis Page Implementation	48
9	Streamlit Dashboard Deployment and Hosting	54
10	Performance Evaluation Metrics	58
11	Challenges and Future Enhancements	59
12	Conclusion	63

List of Figures

Figure No.	Name	Page No.
6.1	Registration Page User Interface	42
7.1	Dashboard Map Interface	47
8.1	Data Table	52
8.2	Analysis Interface	53
9.1	Streamlit Dashboard	56
9.2	Streamlit Dashboard (continued)	57

CHAPTER 1

INTRODUCTION

1.1 Project Motivation

Before the advent of Automatic Meter Reading (AMR) systems, utilities depended heavily on manual methods to track resource consumption. Meter readers had to physically inspect and record values, often facing challenges such as inaccessible meters and wind mills, human error, and delayed data entry. This infrequent and error-prone process led to late billing, customer disputes, and undetected instances of electricity theft. Moreover, fraud detection was minimal, as consumption anomalies often went unnoticed until far too late.

1.2 Project Overview

AMR systems brought a step-change by automating meter data collection, enabling more frequent and accurate monitoring. However, while AMRs can flag certain tampering indicators, they often fall short in predicting theft before it occurs or identifying subtle manipulation trends.

This project bridges that gap by introducing a hybrid solution: using simulated current-based tampering detection and combining it with intelligent forecasting to anticipate future tampering activity. Through machine learning models and a full-fledged dashboard, the system aims to empower utility providers with both visibility and foresight, ultimately reducing non-technical losses and improving operational efficiency.

1.3 Problem Statement

Electricity theft through meter tampering continues to pose a significant challenge for utility providers, especially in developing nations like India. Despite the integration of Automated Meter Reading (AMR) systems, traditional detection

approaches remain reactive, relying on basic threshold flags or periodic manual inspections. These methods often fail to capture subtle, evolving tampering patterns in real-time, leading to substantial financial losses. Moreover, the lack of predictive intelligence and integrated visual monitoring tools hinders timely intervention and strategic planning.

To address these gaps, there is a pressing need for a proactive, data-driven system that not only detects tampering using real-time electrical parameters and AMR event flags but also forecasts future tampering events. Such a solution must offer an interactive and secure dashboard that helps utility providers visualize tampering behaviour, assess meter-level risk, and optimize inspection and prevention efforts.

1.4 Objectives

The main goals of this project are:

- **Simulate Realistic Tampering Scenarios:**
  Create periodic and random tampering patterns based on current variation using tools like Simulink and SCILAB, representing real-world manipulation (e.g., bypassing or partial disconnection).
- **Detect Tampering Using Electrical Parameters and Flags:**
  Use current deviation and AMR-generated event flags (reverse flow, magnetic detection, cover open, etc.) to label tampering behaviour in the data.
- **Forecast Future Tampering Events:**
  Apply ARIMA for time-based forecasting and Random Forest for pattern-based classification to predict high-risk meters.
- **Build an Interactive Monitoring Dashboard:**
  Develop a secure Streamlit-based web dashboard that shows tampering status, predicted risk, meter-wise flags, and geolocation maps.

- **Deploy as a Web-Based System:**
  Host the dashboard online with GitHub integration and provide login protection to allow authorized access by utility staff or admins.
- **Support Utility Decision-Making:**
  Enable better planning of inspections, maintenance, and preventive hardware deployment through insights provided by the system.

1.5 Scope of the Project

This project focuses on designing a software-based solution for detecting and forecasting tampering in smart energy meters. It simulates realistic tampering conditions and leverages machine learning and time-series analysis to identify high-risk meters. The final product includes a full-stack website for utility visualization and decision support.

Inclusions:

- **Tampering Simulation:**
  Simulating both periodic and random tampering events using Simulink and SCILAB, based on realistic current variations and tampering flags.
- **Tampering Detection:**
  Using current thresholds and AMR-generated event flags (reverse current, magnetic field, cover open) to label data points as tampered or normal.
- **Prediction and Forecasting:**
  Implementing ARIMA (for time-based trends) and Random Forest (for multivariate classification) to forecast future tampering.
- **Web Dashboard:**
  An interactive Streamlit-based web application showing meter status, tampering trends, and geographic mapping.

- **Secure Hosting:**
  Hosting the dashboard with login functionality, captcha v3 and redirect from report homepage with session timeout of 15 minutes.

Exclusions:

- **No Real - Time IoT Sensor Integration:**
  The system simulates data; it does not directly read from live smart meters or sensor networks. The data is received from the AMR as an excel or csv.
- **No Field Hardware Deployment:**
  No physical devices are deployed or tested in this project.

1.6 Existing Systems and Literature Review

Existing Systems

Many modern smart meters support tampering detection through event flags, but most are limited to predefined thresholds or single-condition triggers. These systems often lack the ability to learn patterns or forecast future risks.

- **AMR (Automatic Meter Reading):**
  Collects consumption and status data remotely, including tampering events. However, it only detects after tampering occurs - no forecasting or prioritization is performed.
- **Utility Company Tools:**
  Tools like MDAS (Meter Data Acquisition Systems) collect large-scale data, but often lack advanced analytics or predictive capabilities.

Research Gaps Identified

- **Reactive - only Approach:**
  Most current solutions detect tampering after it occurs, with no prediction or early-warning system.

- **No Combined Framework:**
  Lack of systems that integrate detection, prediction, and visualization into a single end-to-end platform.
- **Limited Use of Machine Learning:**
  Most traditional systems use static rules rather than data-driven models that adapt to evolving patterns.

This project addresses these gaps by building a predictive framework that simulates, detects, and forecasts tampering behaviour while visualizing insights through an online dashboard.

CHAPTER 2

SOFTWARE REQUIREMENTS & HARDWARE REQUIREMENTS

2.1 Software Requirements

Simulation and Analysis:

- **MATLAB & Simulink**: Used for modelling and simulating tampering logic based on current values and tampering flags.
- **Scilab**: An open-source alternative to MATLAB, used for simulation of tampering.

Data Analysis and Prediction:

- **Python**: Used for preprocessing, training, and evaluating predictive models.
    - **Libraries Used**:
        - pandas, numpy: Data manipulation and analysis
        - matplotlib, seaborn: Visualization and graph plotting
        - scikit-learn: Machine learning algorithms, including Random Forest
        - statsmodels: Time series forecasting using ARIMA

Web Development:

- **PHP, CSS, JavaScript**: Used for building the front-end of the web dashboard.
- **XAMPP**: Used to locally host the website and manage backend services using Apache and MySQL.
- **Google reCAPTCHA v3:** Integrated for bot protection and security validation on the website.
- **Streamlit**: Used to build one of the interactive web pages for visualizing predictions (ML predictions)
- **Google Maps API**: Used to create an interactive map that visualizes the location of faulty or tampered meters.

- **Bootstrap**: Create websites that work well on mobile devices as well.
- **InfinityFree**: Used for hosting the website with our own domain name.

Miscellaneous Tools:

- **Microsoft Excel:** Used for formatting and storing dataset values.
- **VS Code:** Used as the development environment for writing and testing code.

2.2 Hardware Requirements

Data Collection Hardware (Simulated or Real):

- **Automatic Meter Reading (AMR) Modules**: These are devices integrated into smart energy meters that automatically collect real-time consumption data and transmit it to a central server.
- **Local Server or Cloud Storage**: Used to store historical data and serve it to the prediction models.
- **Internet Connectivity**: Required to stream data to the server and update the dashboard in real time.

Development Machine Specifications:

- **Minimum System Requirements**:
    - RAM: 8 GB or higher
    - Processor: Intel i5 / AMD Ryzen 5 or higher
    - Storage: 500 GB HDD or SSD (SSD recommended for faster performance)

CHAPTER 3

DATA ACQUISITION AND INFRASTRUCTURE SETUP

3.1 SIM Card Configuration and Static IP Allocation

To enable data transmission from remote locations, SIM cards from BSNL, Airtel, and Vi are used. These SIMs are configured with static IP addresses within a predefined range as specified by Tamil Nadu Electricity Board (TNEB). Static IPs are preferred over dynamic ones because they allow consistent and reliable routing of data to the central server. The static IPs are mapped directly to the server room infrastructure for secure and accurate data collection.

3.2 Data Transmission via AMR and Modems

The Automatic Meter Reading (AMR) modules installed in these commercial locations collect real-time data on energy usage and status. This data is passed to modems, where it is digitally encoded into binary format (1s and 0s) for transmission to the central server. This method ensures secure and low-latency communication from remote field devices to the monitoring system.

3.3 Server Infrastructure

The server infrastructure is hosted using HP enterprise-grade devices equipped with Intel Xeon processors. These are chosen over standard consumer-grade processors (e.g., Intel Iris series) for their higher computational power and better multitasking capabilities, essential for managing high volumes of incoming data.
To ensure high availability and fault tolerance, each server is connected to four independent power supplies. In the event of a power supply failure, the remaining ones can seamlessly take over the load. Additionally, two Uninterruptible Power Supplies (UPS) are provided to ensure continuity even in the event of total external power failure.

3.4 Load Balancing and Fault Management

A network load balancer is employed to distribute traffic evenly across multiple servers. Each server is monitored with a status indicator:

- Green indicates healthy and active servers.
- Orange indicates servers that are currently non-functional or experiencing issues.

Even when a server is marked as failed (orange), it remains connected. The load balancer automatically reroutes tasks to other active servers, thereby maintaining service continuity without interruption.

3.5 Data Storage Architecture

The incoming data is stored in high-capacity Solid State Drives (SSDs) and Hard Disk Drives (HDDs) of 600 GB and 1 TB capacities. These enterprise-grade storage devices offer greater speed and storage capability compared to standard PC or laptop drives, ensuring that large volumes of data can be written and accessed efficiently.

3.6 Cache and Data Recovery

Each server is equipped with a 12 MB cache memory. This cache serves as a buffer for temporary data storage and provides quick access to frequently used data. In case of system failures or crashes, the cache can assist in data recovery, minimizing the risk of data loss before it is permanently written to the database.

3.7 Database and File Storage Format

Once the data is received and stored on the server's SSD or HDD, it is organized and saved in structured formats such as Excel sheets (.xlsx), CSV files (.csv), or directly into relational databases. These files contain time-stamped readings, meter IDs, current values, and tampering flags. This structured data format allows for easy analysis, visualization, and further processing using Python scripts, dashboards, or third-party tools.

CHAPTER 4

TAMPERING BEHAVIOUR SIMULATION USING SCILAB AND SIMULINK

4.1 SCILAB Implementation - Real-Time Tampering Detection Simulation

To emulate real-world tampering conditions and validate detection logic before deployment, a simulation was developed using SCILAB's Xcos environment. The goal is to flag tampering events based on abnormal current behaviour under normal voltage conditions.

A. Block Configuration and Parameters

1. Event Clock Generator

- **Period:** 0.00001 seconds (100 kHz sampling rate)
- **Initialization Time:** 0.001 seconds
- **Purpose:** Provides high-frequency timing to simulate real-time signal behaviour and ensure precise response to tampering events.

2. Step Signal for Current Simulation

- **Step Time:** 1 second (simulated tampering initiation point)
- **Initial Value:** 5 A (normal load condition)
- **Final Value:** 0.5 A (simulated current drop indicating potential bypass)
- **Purpose:** Simulates a sudden abnormal change in current typically caused by tampering.

B. Scifunc Block – Core Detection Logic

A custom Scifunc block implements the tampering detection algorithm using simple threshold logic:

```
if u1(1) > 200 & u1(2) < 1 then
   y1 = 1; // Tampering Detected
else
   y1 = 0; // Normal Operation
end
```

- u1(1): Voltage input
- u1(2): Current input
- y1: Output flag (1 = tampering detected, 0 = normal)

C. Signal Flow and Processing

1. Input Signals

- **Voltage Source:** Sinusoidal signal emulating 230V AC mains
- **Current Source:** Step signal mimicking a drop in load current due to tampering
- **MUX Block:** Combines voltage and current into a single input array for the Scifunc block

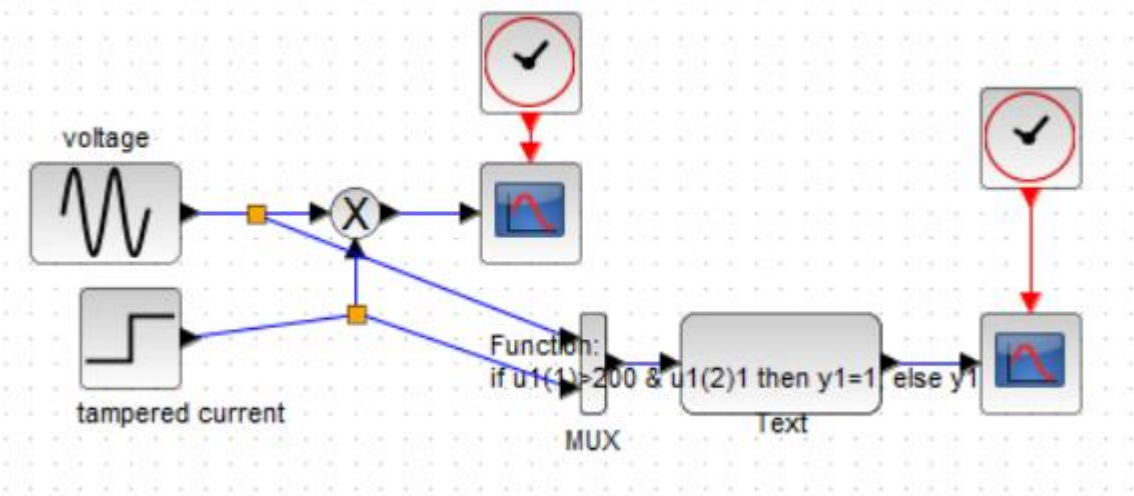
2. Tampering Detection Criteria

- **Voltage Condition:** > 200V — Ensures that tampering is not confused with a power outage
- **Current Condition:** < 1A — Detects possible bypass or manipulation
- **Combined Logic:** Both voltage and current thresholds must be satisfied for a valid tampering flag

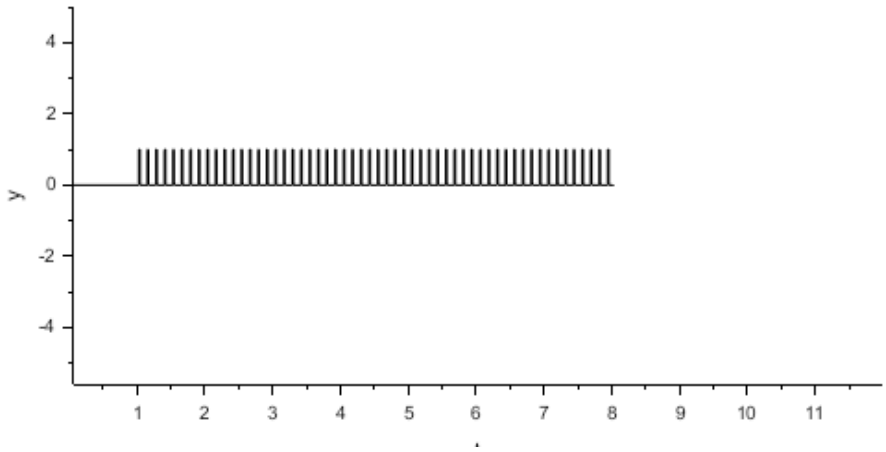
D. Simulation Results and Analysis

- **Tampering Detected at t = 1s**: Output switches from 0 to 1
- **Detection Response Time:** < 0.1 seconds after event
- **Accuracy:** No false positives when voltage drops (i.e., during outages)
- **Output Type:** Binary digital signal

Block Diagram:



Simulation Output:



4.2 SCILAB Implementation - Periodic Tampering Simulation

A. Simulation Architecture

This design introduces a square wave behaviour to current values by periodically reducing the load, simulating tampering at fixed intervals. The setup uses a pulse generator followed by signal manipulation blocks to control current amplitude.

B. Block Configuration and Signal Flow

1. Pulse Generator Block

- **Phase Delay:** 1 second (initial stable period)
- **Pulse Width:** 50% of the cycle (equal ON/OFF periods)
- **Period:** 2 seconds (tampering occurs every 2 seconds)
- **Amplitude:** 1 (standard binary pulse)

2. Signal Flow Configuration

The current signal is synthesized using the following chain:

Pulse Generator → Gain Block (-4.5) → Adder Block (+5) → Current Output

- **Gain Block:** Scales pulse output to simulate current reduction.
- **Adder Block:** Adds the reduced signal to the baseline current.

C. Current Signal Model

Current Output Calculation:

- **Normal Mode (Pulse = 0):**
  - Current = 5 + (-4.5 × 0) = **5.0 A**
- **Tampering Mode (Pulse = 1):**
  - Current = 5 + (-4.5 × 1) = **0.5 A**

This behaviour simulates intermittent bypass, where the current drops drastically during specific intervals while voltage remains unaffected.

Periodic Behaviour Timeline:

Time Interval	Pulse Value	Current Output	Condition
0–1 s	0	5.0 A	Normal
1–2 s	1	0.5 A	Tampered
2–3 s	0	5.0 A	Normal
3–4 s	1	0.5 A	Tampered

D. Block Parameter Significance

Gain Block (-4.5):

- Scales the pulse to simulate a 4.5 A drop, reducing current by 90%.
- Ensures consistent tampering depth during each pulse cycle.

Constant Block (+5):

- Defines the nominal operating current.
- Keeps simulation baseline consistent for reliable detection benchmarking.

```
if u1(1) > 200 & u1(2) < 1 then
   y1 = 1; // Tampering Detected
else
   y1 = 0; // Normal Operation
end
```

- u1(1): Voltage input
- u1(2): Current input
- y1: Output flag (1 = tampering detected, 0 = normal)

E. Simulation Results Analysis

Output Signal Characteristics:

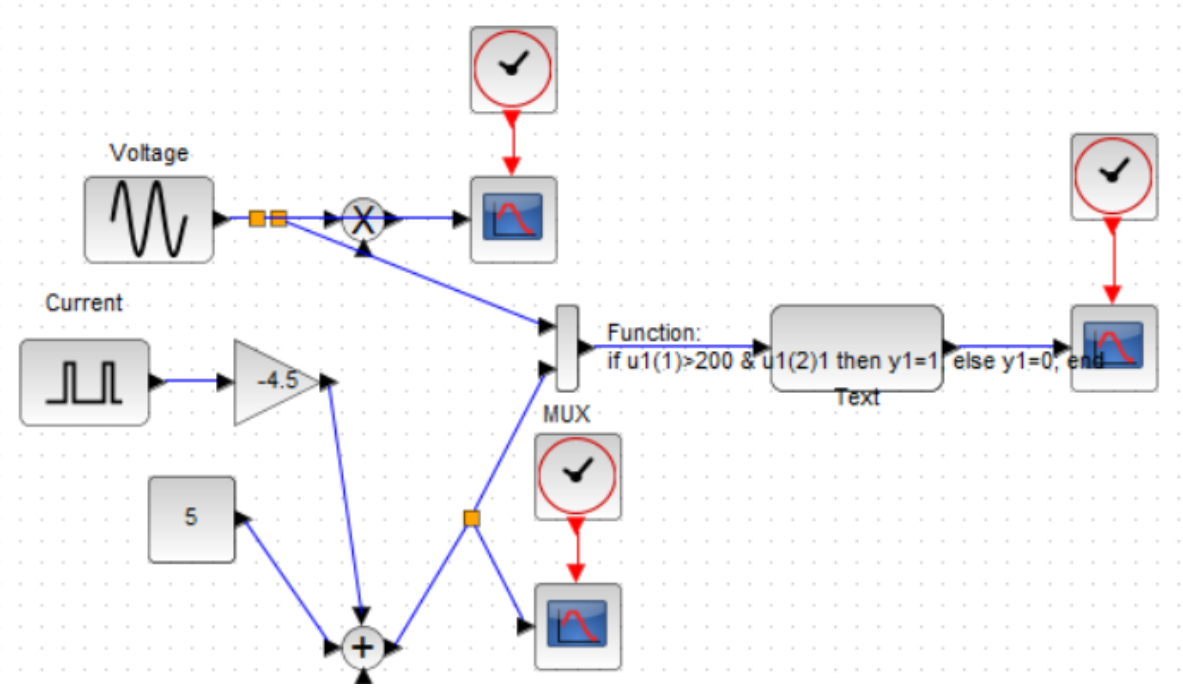
- **Waveform Type:** Square wave oscillating between 5.0 A and 0.5 A
- **Amplitude Range:** 4.5 A variation
- **Duty Cycle:** 50% (1s ON, 1s OFF)
- **Frequency:** 0.5 Hz (2s cycle period)

Tampering Detection Response:

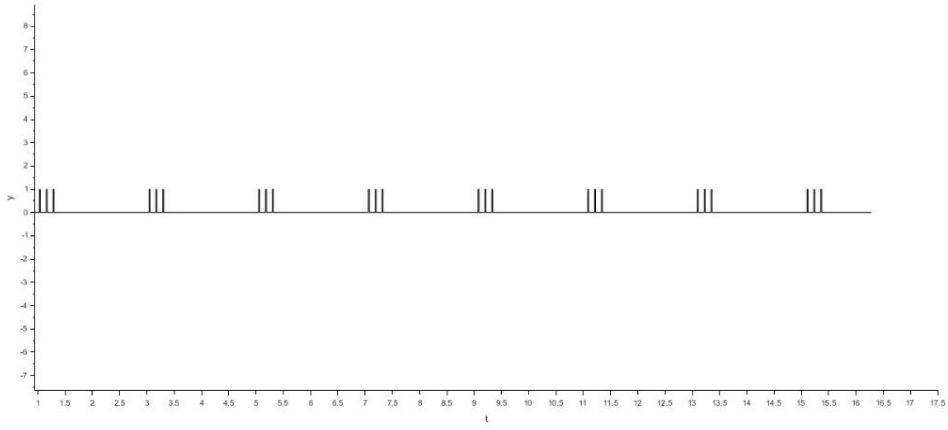
- **Voltage Check:** u1(1) > 200 V (indicating grid power supply is active)
- **Current Threshold:** u1(2) < 1 A (interpreted as tampering)

- **Detection Output:** Binary flag (y1 = 1 when tampering, else y1 = 0)

Block Diagram:



Simulation Output:



4.3 SCILAB Implementation - Random Current-Based Tampering Simulation

A. Simulation Architecture Overview

- This setup introduces randomly generated current values in a controlled environment to simulate both normal and tampered load conditions.

- Tampering is inferred when the current drops below a defined threshold while the voltage remains normal, mimicking energy theft through partial bypass.

B. Block Configuration and Parameters

1. Random Generator Block

- **Distribution Type:** Uniform (flag = 0)
- **Current Range:** 0 A to 32 A
  - **Parameter A:** 0
  - **Parameter B:** 32
- **Seed Values:** [2113248, 7560437] (for reproducibility)
- **Data Type:** Real double (1)
- **Purpose:** Generates random current values to simulate real-world fluctuations in load and tampering events.

2. Event Clock Generator

- **Period:** 0.1 seconds (10 Hz sampling rate)
- **Initialization Time:** 1 second
- **Purpose:** Allows time-based monitoring and response.

C. Tampering Detection Logic

Detection is based on current thresholds applied to the random samples:

```
if u1(1) > 200 & u1(2) < 9.6 then
   y1 = 1; // Tampering detected
else
   y1 = 0; // Normal
end
```

- **Voltage Condition (u1(1)):** Must exceed 200V to confirm supply is active.
- **Current Condition (u1(2)):** If current < 9.6A (30% of 32A capacity), tampering is flagged.

D. Current Analysis and Probabilistic Modelling

1. Threshold Zones

- **Normal Range:** 15–32 A (typical residential/commercial use)
- **Suspicious Range:** 9.6–15 A (borderline behaviour)
- **Tampering Range:** 0–9.6 A (high tampering likelihood)

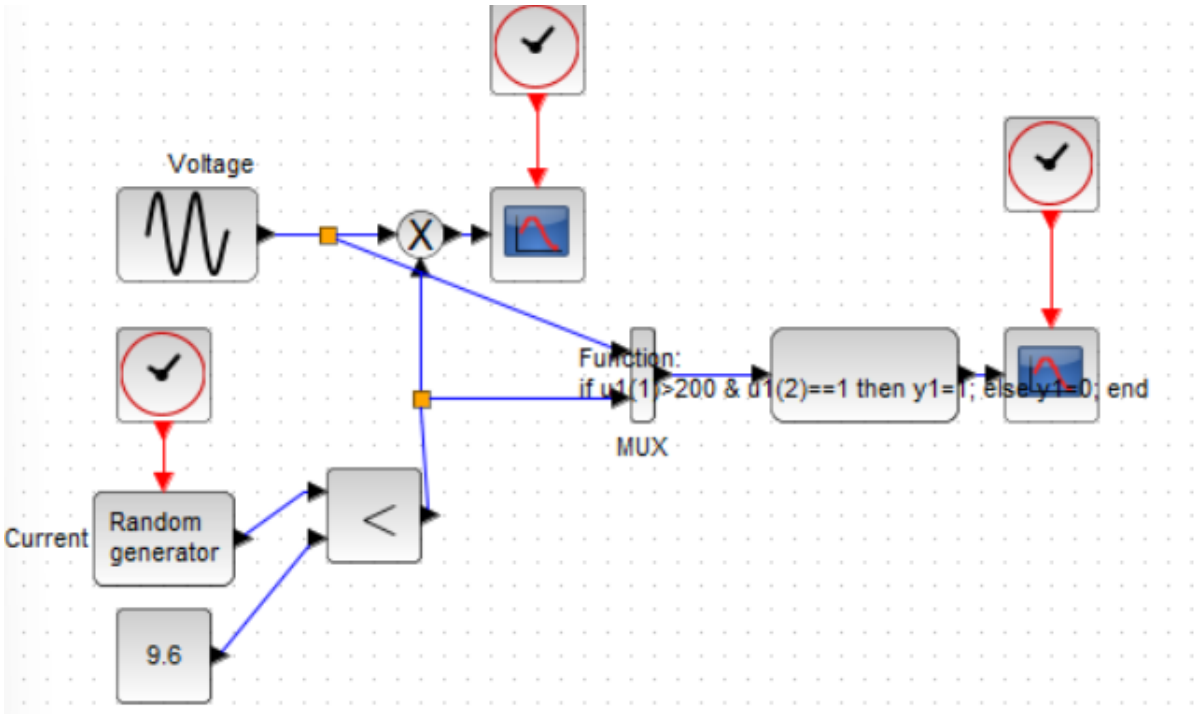
2. Probability Metrics

- **P(Current < 9.6 A):** 30% — Expected tampering probability
- **P(Current ≥ 9.6 A):** 70% — Normal operation
- **Tampering Event Frequency:** Approximately 3 out of every 10 samples

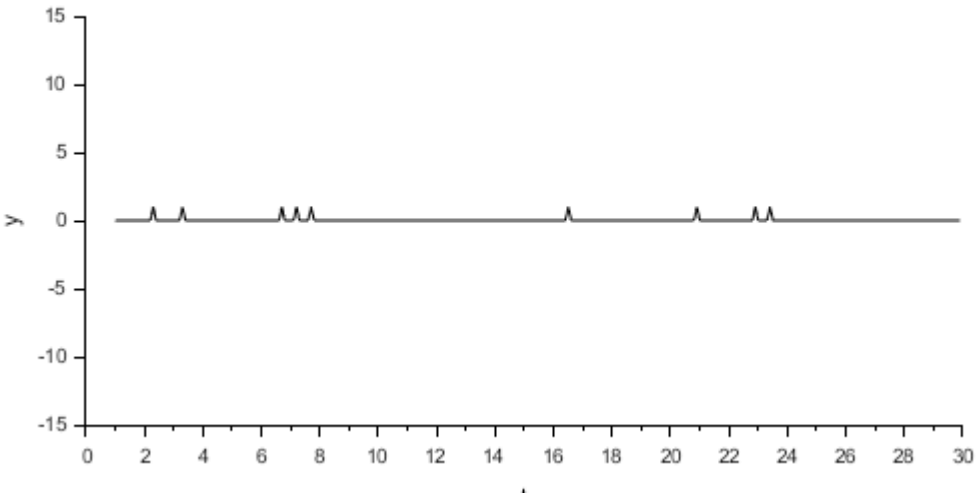
E. Simulation Workflow

1. **Initialization at t = 1s:** Clock begins activating random current generation.
2. **Sampling:** A new current value is generated every 0.1s using uniform distribution.
3. **Logic Check:** If voltage is normal and current is low, a tampering flag is raised.
4. **Output Generation:** y1 = 1 for tampering; y1 = 0 otherwise.
5. **Continuous Monitoring:** Binary output.

Block Diagram:



Simulation Output:



4.4 SIMULINK - Advanced Tampering Detection Implementation

This section presents a multi-parameter tampering detection system modelled in Simulink using real AMR data imported from an Excel sheet. The system evaluates both physical and electrical tampering indicators to enhance detection accuracy under realistic conditions.

A. System Architecture Overview

The simulation integrates a dual-layer tampering detection algorithm that processes real-time signal inputs from a meter, evaluates anomalies, and generates binary tampering labels based on a comprehensive rule set.

B. Input Configuration

1. Data Source

- **File:** datasheet.xlsx, Sheet1
- **Sampling:** Discrete, hourly resolution

2. Signal Mapping

```
% Input Vector Mapping (u):
u(1) = voltage                  % Simulated 230V AC
u(2) = consumption_kWh          % Energy use per hour
u(3) = load_variance            % Variance in consumption patterns
u(4) = reverse_current          % Power flowing backward
u(5) = magnetic_field_detected  % External magnetic presence
u(6) = cover_open_event         % Meter tamper/access
u(7) = neutral_missing_event    % Neutral cut or shift
u(8) = voltage_anomaly          % High/low voltage tampering
u(9) = communication_loss       % AMR signal disruption
```

Note: The voltage input is generated as a sine wave with **amplitude 311V**, which is the **peak of a 230V RMS signal** (i.e., 230×√2).

C. Detection Function – MATLAB Code

The core tampering decision logic is implemented in the following embedded function block:

```
function tampering_label = detect_tampering_simple(u)
    % Extract signals
    voltage = u(1);
    consumption_kWh = u(2);
    load_variance = u(3);
    reverse_current = u(4);
    magnetic_field_detected = u(5);
    cover_open_event = u(6);
    neutral_missing_event = u(7);
    voltage_anomaly = u(8);
    communication_loss = u(9);

    % Calculate current (1-hour interval)
    time_interval_hours = 1;
    power_watts = (consumption_kWh * 1000) /
time_interval_hours;
    calculated_current = power_watts / 230;

    % Thresholds
    threshold_current = 2.0;
    threshold_variance = 0.4;

    % Tampering Logic
    if ((abs(voltage) > 220) && ...
        (reverse_current == 1 || magnetic_field_detected == 1 ||
cover_open_event == 1))
        tampering_label = 1;
    elseif ((neutral_missing_event == 1 || voltage_anomaly == 1
|| communication_loss == 1) && ...
            (calculated_current < threshold_current || load_variance
> threshold_variance))
        tampering_label = 1;
    else
        tampering_label = 0;
    end
```

D. Detection Logic Overview

1. Primary Tampering Detection:

- **Trigger:** Voltage present and at least one of the physical tamper events active
- **Reason:** Magnetic field interference, cover opening, or reverse flow indicate external tampering

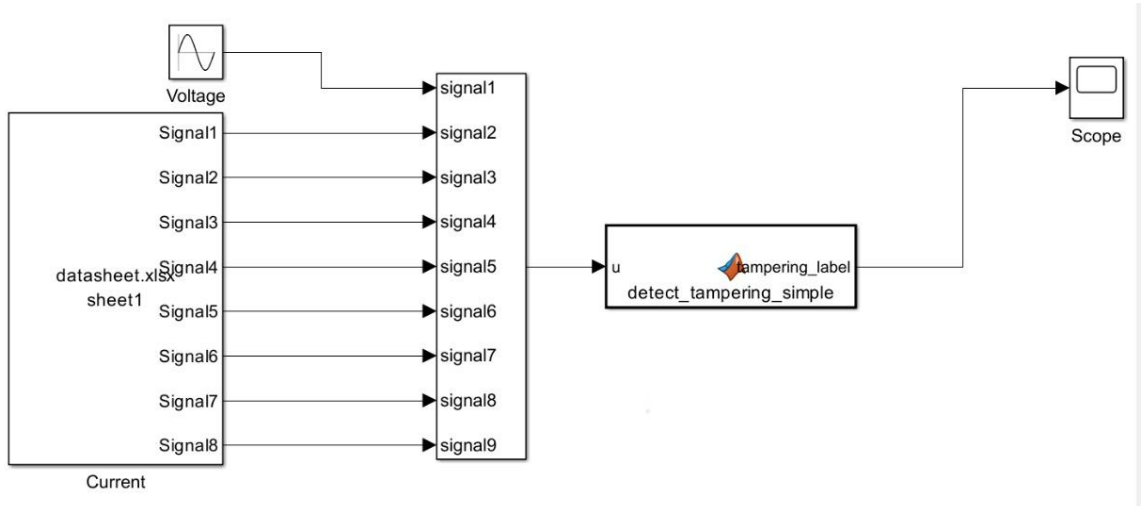
2. Secondary Tampering Detection:

- **Trigger:** Any electrical anomaly with suspicious consumption (low current or unstable load)
- **Reason:** Covers hidden forms of theft like neutral bypass or communication disruption

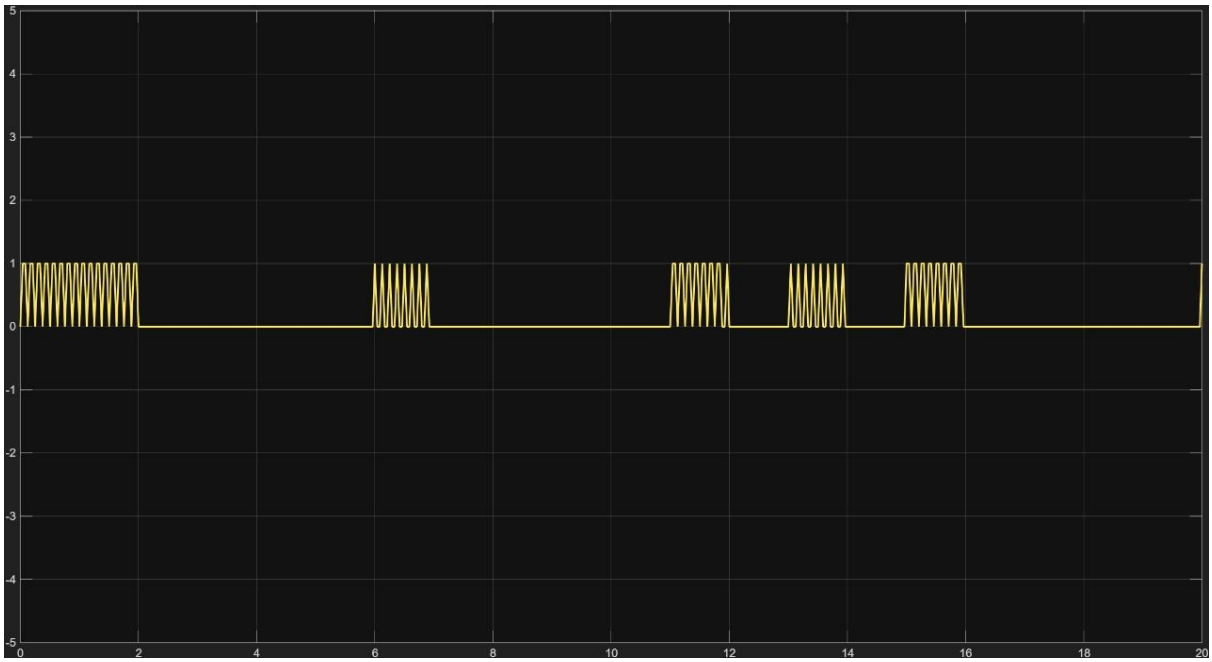
E. Simulation Results & Output

- **Waveform Output:** Binary flags (0 or 1) showing tamper status over time
- **Event Frequency:** Matches real patterns of intermittent and clustered tampering
- **Detection Shape:** Sharp rising edges, stable tampering windows, no flickering

Block Diagram:



Simulation Output:



4.5 Summary

The advanced tampering detection simulation using SIMULINK and SCILAB was developed to analyse real-world meter data and identify fraudulent activity based on a wide set of parameters. This simulation was essential to bridge the gap between theoretical detection logic and actual field scenarios, allowing us to test under practical constraints like fluctuating voltage, consumption variance, and physical/electrical tampering.

By leveraging multi-parameter input from an Excel-based AMR dataset, we implemented a two-tiered detection system, prioritizing immediate physical tampering while also analysing subtle electrical anomalies through computed current and variance thresholds.

CHAPTER 5

TAMPERING PREDICTION AND FORECASTING

5.1 Rationale for Tampering Prediction

Electricity meter tampering leads to substantial energy theft and infrastructure instability, especially in dense residential and semi-urban networks. While detection techniques using thresholds and event-based rules (like low current, magnetic influence, or reverse power flags) are useful, they are fundamentally reactive in nature. Tampering often occurs intermittently, making it difficult to capture with static thresholds or rule-based logic alone.

To address this limitation, we adopt a predictive approach, aiming to forecast tampering behaviour in advance using machine learning models. This allows for:

- **Early Intervention**: Identifying meters likely to be tampered with in the near future.
- **Prioritized Inspection**: Enabling targeted audits of high-risk meters instead of random field visits.

In our project, we use two modelling approaches:

- **ARIMA (AutoRegressive Integrated Moving Average)**: For capturing time-series patterns in tampering flags.
- **Random Forest Classifier**: For learning non-linear relationships between multiple features (e.g., consumption behaviour, event flags) and tampering risk in different meters.

5.2 Dataset Structuring and Feature Engineering

The prediction task relies on a structured dataset compiled from operational data exported from AMR-based Excel files. Each record represents a meter reading at a specific timestamp, along with associated features and event flags.

Data Transformation

- Missing values (especially in communication status) are handled using forward fill and linear interpolation.
- Data is resampled to daily frequency to align with ARIMA requirements.
- Categorical variables (consumer type, location) are label-encoded for compatibility with Random Forest model.

Train-Test Partitioning

To simulate real deployment:

- Training is done on historical data (2023).
- Testing is done on newer, unseen data (2024).
  This ensures the model generalizes to future tampering events rather than memorizing patterns.

5.3 Implementation of Forecast and Prediction

The forecasting and tampering prediction system is implemented using Python, combining statistical time series modelling and machine learning. The process involves loading and preprocessing meter data, training predictive models, forecasting future consumption, and finally predicting tampering risk for each meter.

Key Components

- **Time Series Forecasting (ARIMA):** The ARIMA model is used to predict 2024 consumption per meter based on historical daily energy usage. Each meter is modelled independently to capture local trends and seasonality.
- **Classification (Random Forest):** A Random Forest classifier is trained using labelled 2023 data.

5.4 Results and Visualizations

The dashboard provides both qualitative insights and quantitative forecasts for 2024 tampering risks. Major components include:

Top Predictive Features

The Random Forest model identifies key predictors of tampering. The most important features include:

- reverse_current
- load_variance
- efficiency_score
- magnetic_field_detected
- peak_usage_shift

These features often correlate strongly with manipulation patterns in meter behaviour.

Tampering Correlation Matrix

A heatmap visualization reveals the correlation between various engineered features and tampering labels. Positive correlation with reverse_current, and negative correlation with load_factor, supports domain expectations. This shows that higher reverse_current values align with increased tampering likelihood unlike load_factor in which elevated load_factor correlates with reduced tampering.

Tampering Alerts

Predictions for 2024 are displayed per meter, including:

- Meter ID
- Timestamp
- Forecasted consumption
- Tampering prediction flag (1 for tampering)

Alerts are downloadable as CSV, allowing utilities to plan inspection visits or remote deactivation actions.

Consumption vs Tampering Plot

A scatter plot helps visualize the separation between tampered and normal readings in terms of consumption and load variance, supporting model interpretability.

5.5 Discussion and Evaluation

Model Performance (2023 Test Set)

The Random Forest classifier, evaluated on a held-out 2023 test set, achieved strong classification metrics:

- **Precision and Recall:** High values in both indicate balanced detection capability.
- **F1-score:** Indicates the model avoids false positives/negatives effectively, essential for minimizing unnecessary inspections.

Robustness of Forecasting

The ARIMA-based consumption forecast for 2024 performed reliably for the majority of meters. Forecast errors due to missing or highly irregular data were handled using interpolation.

Advantages

- **Proactive Detection:** Instead of waiting for tampering to be detected passively, future risks are identified early.
- **Meter-Level Resolution:** Each meter is modelled and analysed individually.
- **Scalability:** The system is scalable to thousands of meters with minor adjustments.
- **Interpretability:** Feature importances and correlation matrices support transparent decision-making.

Limitations

- ARIMA may fail for non-stationary or sparse data.
- Prediction accuracy may degrade if data quality is poor.

CHAPTER 6

LOGIN PAGE IMPLEMENTATION

This section details the implementation of register.php, the core authentication interface for the TNEB project dashboard. It serves as a unified entry point for both new user registrations and existing user logins, prioritizing secure and efficient access to sensitive meter data and analytics. The design incorporates robust security measures to safeguard against automated threats and unauthorized access, ensuring a reliable user authentication experience.

6.1 Overview and Purpose

The register.php script functions as the primary authentication gateway for the TNEB project dashboard. Its dual purpose encompasses user registration and login authentication. The script is engineered to facilitate secure and user-friendly access to crucial meter data and analytical insights, while integrating strong protective measures against automated attacks (bots) and unauthorized entry.

6.2 Architecture and Technologies

The implementation of register.php leverages a multi-tiered architecture:

- Front-End: Developed using HTML, CSS, and vanilla JavaScript, the front-end provides a responsive and intuitive user interface. It manages form handling and dynamically toggles between registration and login views, enhancing user experience.
- Back-End: PHP drives the server-side logic, overseeing form data processing, session management, and secure interactions with the database.
- Database: MySQL is employed for the secure storage of user credentials, utilizing robust password hashing for data integrity.
- Security: Integration of Google reCAPTCHA Enterprise v3 offers invisible bot protection.

- Google Cloud Platform: This platform hosts the reCAPTCHA Enterprise services, providing critical API-based risk analysis capabilities.

6.3 Registration and Login Workflow

Registration Process:

- Users are prompted to provide a unique username and confirm a matching password.
- Passwords are securely hashed using PHP's password_hash() function prior to database storage.
- The system performs a crucial check for duplicate usernames to prevent conflicts.
- Upon successful registration, the user's data is persisted in the database, and a secure session is immediately initiated, granting access to the dashboard.

Login Process:

- Users submit their credentials (username and password).
- Inputs are sanitized to prevent injection attacks.
- The provided password is verified against the stored hash using password_verify().
- Successful authentication leads to session creation and redirection to the dashboard. Authentication failures trigger clear and informative error messages to the user.

6.4 Google reCAPTCHA Enterprise v3 Integration

- **Purpose:** Google reCAPTCHA Enterprise v3 provides an invisible and seamless bot protection mechanism. It analyses user behaviour in real-time, assigning a risk score without requiring explicit user interaction, thereby preserving usability and enhancing security.

Implementation Details:

- **Client-Side Mechanism:** The JavaScript snippet loads the reCAPTCHA Enterprise script (https://www.google.com/recaptcha/enterprise.js). On page load or when a user toggles between the login/register forms, the verifyRecaptchaEnterprise() function is called with the configured RECAPTCHA_SITE_KEY and the appropriate action (e.g., 'register' or 'login'). This communicates with Google's reCAPTCHA service to generate a unique, action-specific token. This token is then dynamically injected into a hidden input field within the respective HTML form (e.g., <input type="hidden" id="g-recaptcha-response-register" name="g-recaptcha-response">).

- **Server-Side Validation:** When the form is submitted, the PHP verifyRecaptchaEnterprise($token, $siteKey, $action) function sends the received token, the configured site key, and the expected action to the Google reCAPTCHA Enterprise API. The API processes this information and returns a JSON response containing a risk analysis, including a riskAnalysis.score (ranging from 0 to 1). The server-side script then validates if the score meets a predefined threshold (e.g., ≥0.5 a standard value as recommended by Google) and if the expectedAction matches the action received. Failure to meet these criteria results in the blocking of the submission, effectively preventing automated attacks.

- **Protection Efficiacy:** This method offers a user-transparent defense against bots, credential stuffing, and other automated malicious activities, significantly enhancing the security posture of the authentication process.

6.5 Security Measures and Best Practices

- **Password Hashing:** Passwords are never stored in plain text. Instead, they are robustly hashed using PHP's password_hash($password,

PASSWORD_DEFAULT) function. PASSWORD_DEFAULT ensures the use of the strongest available hashing algorithm (currently bcrypt), which is designed to be slow and resistant to brute-force attacks. For verification, password_verify($password, $hashed_password) is used to securely check if a provided plain-text password matches a stored hash, safely performing the comparison without re-hashing the stored value and protecting against timing attacks.

- **Input Sanitization:** User inputs are meticulously sanitized using mysqli_real_escape_string($con, $input) before they are incorporated into database queries. This function escapes special characters, thereby neutralizing potential SQL injection attacks that could lead to unauthorized data access or manipulation.

- **Session Management:** Secure PHP sessions (session_start()) are utilized to maintain authenticated user state. Upon successful authentication, crucial user-specific information such as $_SESSION['user_id'], $_SESSION['username'], and $_SESSION['logged_in'] are securely stored within the session. These variables allow the system to identify and maintain the authenticated state of the user across different pages of the application, enabling restricted access to protected resources.

- **Error Handling:** The script includes explicit error handling for various scenarios, such as: a username already existing (checked via mysqli_errno($con) == 1062 for duplicate entry) during registration, passwords not matching ($password !== $confirm_password) during registration, incorrect password (password_verify() failure) during login, username not found (no result from mysqli_query()) during login, and reCAPTCHA verification failures. These errors are captured and displayed as clear, concise messages above the respective forms, providing immediate and actionable feedback to the user regarding the reason for their failed attempt.

- **Form Submission Controls:** JavaScript is implemented to disable the submit button upon form submission (btn.disabled = true;), preventing accidental multiple requests. It also ensures that a fresh reCAPTCHA token is generated (setRecaptchaRegister() or setRecaptchaLogin()) before each submission, enhancing security.

6.6 User Interface and Experience Design

- **Responsive Layout:** The page features a centered, card-based design, presenting a clean and modern aesthetic. It includes a prominent TNEB logo and clear form titles.
- **Dynamic Form Toggling:** Users can seamlessly switch between registration and login forms without requiring a page reload. This is achieved through intuitive links (onclick="toggleForm('login')" / onclick="toggleForm('register')") and JavaScript-driven UI updates (classList.add('hidden'), classList.remove('hidden')), improving fluidity.
- **Input Validation:** Client-side HTML attributes enforce constraints on input fields, such as minlength="6" for passwords and maxlength="32" for usernames, guiding users to provide valid data before server-side processing.
- **Error and Success Messaging:** Feedback is provided through inline error messages displayed prominently above the form (...), offering immediate and clear communication to the user.
- **reCAPTCHA Badge Styling:** The reCAPTCHA badge is styled to be minimally intrusive (.grecaptcha-badge CSS) while remaining visible and compliant with Google's terms of service.

6.7 Integration with the Dashboard

- **Access Control:** Only users with a valid and active session (i.e., authenticated users, indicated by $_SESSION['logged_in'] = true;) are successfully redirected to the protected dashboard (default.php).

- **Seamless Navigation:** The login system acts as the primary line of defense, ensuring that all subsequent interactions, data access, and operations within the dashboard are performed by verified users.
- **Modularity:** The architectural design of register.php allows for straightforward future enhancements, such as the integration of multi-factor authentication (MFA) or OAuth.

6.8 Admin User Database Structure (phpMyAdmin Implementation)

The admin authentication and management system is underpinned by a dedicated MySQL database named admins, created and managed using phpMyAdmin. Within this database, the primary table for storing admin user credentials and related information is named users.

Table Structure: users

The users table is designed to store all essential information for each admin account.

id	username	password
0	krishna	$2y$10$V3Jtm.l3ja8nnXfpWooxHenWxKA4l6TmbT4ez1F.7Xp...
0	user	$2y$10$9VXE.5KbuzhDsZjNEto/jO3n1Y8B1AtOWkql5e8dhg1...
0	user1	$2y$10$6aOBfXz6vRLhLD1fJ./R2ulAcPdsAvkb0S0R7mf5Ruo...
0	anusuya	$2y$10$xsHGo17DFzV7r8cqCwlUQeMrEaO9CaPeuUSfl0EgVkN...
0	nishanthi	$2y$10$q/1.Hij7f4MNj/jfXosk6OBD40m8cvoGcjg85nBDxpp...

Field Name	Data Type	Attributes	Description
id	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for each admin user
username	VARCHAR(255)	UNIQUE, NOT NULL	Admin's login name (must be unique)
password	VARCHAR(255)	NOT NULL	Hashed password for secure authentication

Security Practices

Password Hashing: Passwords are never stored in plain text. Instead, they are hashed using secure algorithms like bcrypt, which is recommended for modern PHP applications. This protects admin credentials even in the event of a database breach.

Unique Usernames: The username field is set as unique to prevent duplicate entries and ensure each admin can be reliably identified and authenticated.

phpMyAdmin Management: All table creation, editing, and record management are performed through phpMyAdmin's graphical interface, allowing for easy administration, field modifications, and direct record browsing.

Example: Table Creation SQL

```
CREATE TABLE users (
  id INT AUTO_INCREMENT
PRIMARY KEY,
  username VARCHAR(255) NOT NULL
UNIQUE,
  password VARCHAR(255) NOT NULL
);
```

This structure provides a robust foundation for admin authentication and access control within the application, aligning with best practices for security and maintainability.

Usage in the Application

- During registration, admin passwords are hashed and stored in the password field.
- During login, submitted passwords are verified against the stored hash using secure verification functions.
- Only authenticated admins are granted access to sensitive dashboard and management features, ensuring robust access control.
- By adhering to these design and security principles, the admins database and users table implemented via phpMyAdmin provide a secure and reliable backbone for admin management within the system.

6.9 Registration Page User Interface

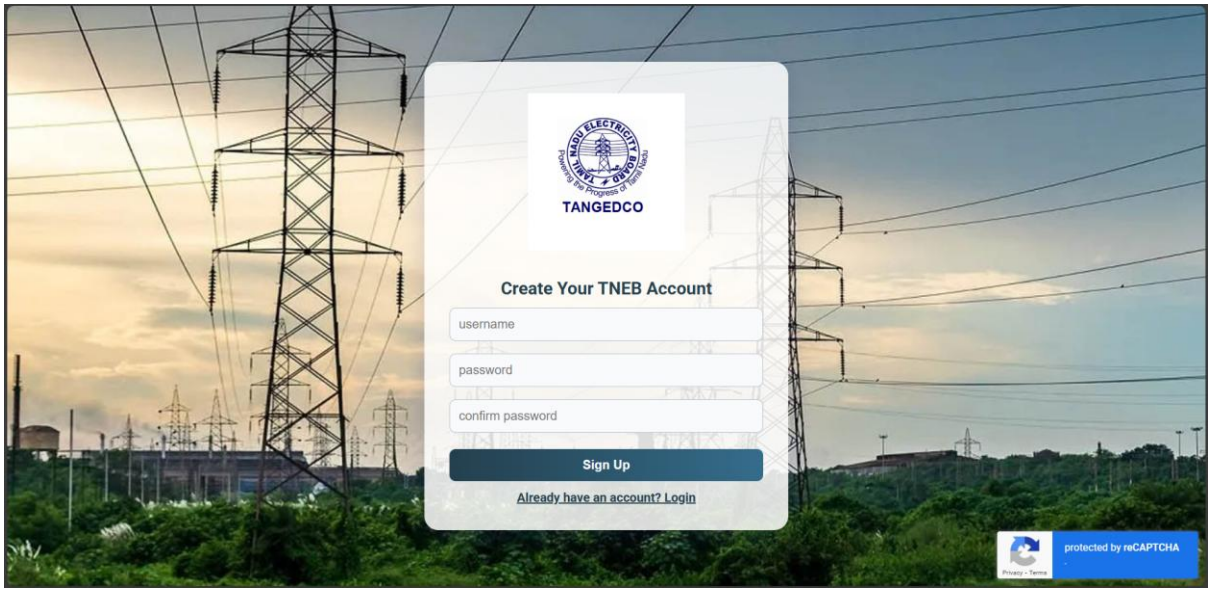


Fig. 6.1

CHAPTER 7

DASHBOARD PAGE IMPLEMENTATION

7.1 Overview and Purpose

The dashboard page is the main interface for authenticated users, providing a real-time, interactive map of AMR SIM meter locations, their operational statuses, and tampering alerts. It is designed to support efficient monitoring, quick access to critical information, and secure user management at all times.

7.2 User Authentication, Session Management, and Logout Option

- Session Enforcement: Only authenticated users with an active session can access the dashboard. PHP sessions are used to track login status, and a 15-minute inactivity timeout is enforced for security. If the session expires, the user is redirected to the login page.
- Profile Dropdown and Logout: The top navigation bar includes a user profile dropdown menu, which displays the logged-in username and provides a clearly labeled "Logout" option. Selecting "Logout" immediately destroys the user's session and redirects them to the login page, ensuring that all sensitive data is protected and access is revoked upon user request.
- Logout Implementation: The logout process is handled by a dedicated script (e.g., logout.php), which calls session_unset() and session_destroy() in PHP, followed by a redirect to the login interface. This approach ensures a clean session termination and prevents unauthorized access if the browser remains open.

7.3 Google Maps API Integration

- The dashboard uses the Google Maps JavaScript API to render a dynamic map centered on Tamil Nadu, providing a geospatial overview of all AMR SIM meters.

- Each meter is represented by a marker placed at its latitude and longitude from the database.
- Marker icons visually distinguish between normal (default icon), non-reporting (red icon), and tampered meters (alert in info window).
- Clicking a marker opens an info window with detailed meter information: name, status, location, SIM IP, tampering status, and reason if available.
- The map automatically adjusts its bounds to ensure all markers are visible, regardless of the number of meters.

7.4 Weather and Real-Time Clock Display

- The top navigation bar includes a weather indicator showing static data for Chennai (e.g., “36°C, overcast”), which can be extended to fetch live weather updates from external APIs for future enhancements.
- A live digital clock is implemented using JavaScript, displaying the current IST time and updating every second to provide users with real-time context.

7.5 Map Visualization and Data Display

- All meters retrieved from the database are displayed as markers on the map, with icons and info windows reflecting their current status and alert level1.
- Info windows for each marker provide:
    - Meter name
    - Status (Reporting/Non-Reporting)
    - Geographic coordinates
    - SIM IP address
    - Tampering status and reason (if applicable)
    - Color-coded alerts for critical conditions.
    - The map’s viewport automatically fits all markers, ensuring no data is hidden off-screen.

7.6 Navigation and User Controls

- A prominently styled “Report” button in the dashboard header allows users to navigate to the tabular report view (index.php), where detailed meter data is presented in a table for further analysis.
- The profile dropdown in the top bar provides access to user information and a secure logout option, supporting easy session management and user awareness.
- The dashboard layout is fully responsive, adapting to various screen sizes for optimal usability on both desktop and mobile devices.

7.7 Security and UI Best Practices

- Strict session controls and inactivity timeouts are enforced to prevent unauthorized access.
- All user-facing data is sanitized before display to prevent cross-site scripting (XSS) attacks, and database connections are securely managed.
- Branding elements, such as the TNEB logo, are enhanced with interactive animations, providing a professional and engaging user experience.

7.8 Map Database Structure: map_data and meter_data

The dashboard’s map visualization is powered by a MySQL database named map_data, with the primary table for storing all AMR SIM meter information titled meter_data.

Table Structure and Key Fields

- The meter_data table is designed to capture every essential attribute needed for geospatial mapping and operational monitoring. The main fields include:
- loc_id: Unique identifier for each meter location (Primary Key, auto-incremented).
- name: Name of the location or meter.
- latitude & longitude: Geographic coordinates for precise map placement.
- color: Used for marker color coding on the map.

- status: Operational status (e.g., Operational, Non Reporting).
- date: Last update date for the meter data.
- Capacity: Meter capacity (e.g., 100kW, 250kW).
- SIM_IP: SIM card IP address used for network communication.
- tampering_label: Indicates if tampering is detected (0 = Normal, 1 = Tampered).
- tampering_reason: Describes the reason for tampering, if any.
- This schema supports all core map features, including marker placement, color coding, and real-time tampering alerts.

Data Usage in the Dashboard

- The dashboard queries the meter_data table to retrieve all meter locations and operational details for real-time visualization using the Google Maps API.
- Markers are dynamically assigned based on the status and tampering_label fields, allowing users to quickly distinguish between normal, non-reporting, and tampered meters.
- Info windows on the map display all key attributes for each meter, such as location, status, SIM IP, and tampering information.

COL 1	COL 2	COL 3	COL 4	COL 5	COL 6	COL 7	COL 8	COL 9	COL 10	COL 11
loc_id	name	latitude	longitude	color	status	date	Capacity	SIM_IP	tampering_label	tampering_reason
0	Ariyalur	11.1372	79.0917	green	Operational	2025-06-01	100kW	172.18.0.0	0	Normal
1	Ariyalur	11.1372	79.0917	green	Operational	2025-06-01	100kW	172.18.0.1	0	Normal
2	Ariyalur	11.1372	79.0917	green	Operational	2025-06-01	50kW	172.18.0.2	0	Normal
3	Ariyalur	11.1372	79.0917	green	Operational	2025-06-01	200kW	172.18.0.3	0	Normal
4	Ariyalur	11.1372	79.0917	green	Operational	2025-06-01	50kW	172.18.0.4	0	Normal
5	Chengalpattu	12.688	79.9764	green	Operational	2025-06-01	250kW	172.18.0.5	0	Normal
6	Chengalpattu	12.688	79.9764	green	Operational	2025-06-01	100kW	172.18.0.6	0	Normal
7	Chengalpattu	12.688	79.9764	green	Operational	2025-06-01	50kW	172.18.0.7	0	Normal
8	Chengalpattu	12.688	79.9764	green	Operational	2025-06-01	250kW	172.18.0.8	0	Normal
9	Chengalpattu	12.688	79.9764	green	Operational	2025-06-01	100kW	172.18.0.9	0	Normal
10	Chennai	13.0827	80.2707	green	Operational	2025-06-01	200kW	172.18.0.10	0	Normal
11	Chennai	13.0827	80.2707	red	Tampered	2025-06-01	100kW	172.18.0.11	1	Neutral Disconnect
12	Chennai	13.0827	80.2707	green	Operational	2025-06-01	150kW	172.18.0.12	0	Normal
13	Chennai	13.0827	80.2707	green	Operational	2025-06-01	50kW	172.18.0.13	0	Normal
14	Chennai	13.0827	80.2707	green	Operational	2025-06-01	50kW	172.18.0.14	0	Normal
15	Coimbatore	11.0168	76.9558	green	Operational	2025-06-01	250kW	172.18.0.15	0	Normal
16	Coimbatore	11.0168	76.9558	green	Operational	2025-06-01	150kW	172.18.0.16	0	Normal
17	Coimbatore	11.0168	76.9558	green	Operational	2025-06-01	150kW	172.18.0.17	0	Normal
18	Coimbatore	11.0168	76.9558	green	Operational	2025-06-01	250kW	172.18.0.18	0	Normal
19	Coimbatore	11.0168	76.9558	red	Tampered	2025-06-01	250kW	172.18.0.19	1	Communication Failure
20	Cuddalore	11.75	79.75	green	Operational	2025-06-01	250kW	172.18.0.20	0	Normal

7.9 Dashboard Map Interface

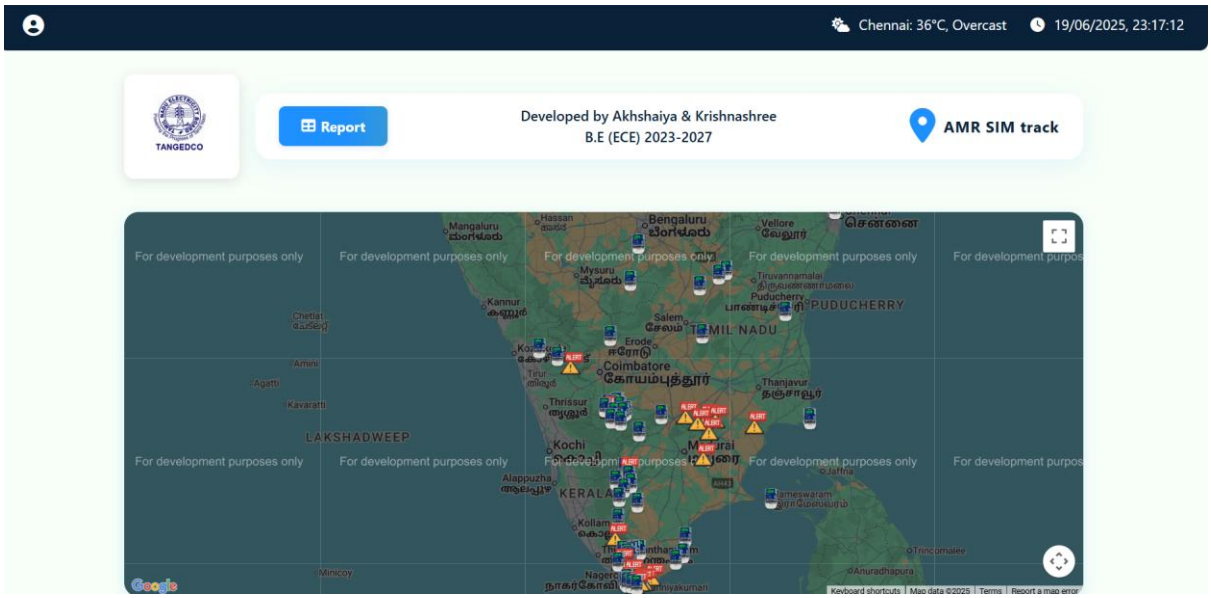


Fig. 7.1

CHAPTER 8

DATA TABLE AND ANALYSIS PAGE IMPLEMENTATION

8.1 Overview and Purpose

The index.php page serves as the data-centric interface for authenticated users, presenting a comprehensive, filterable table of meter data alongside interactive analysis tools. This page is designed to support detailed inspection, event analysis, and operational decision-making by providing advanced filtering, visual summaries, and seamless navigation to external dashboards.

8.2 Data Filtering Functionality

- The top of the page features a filter form allowing users to refine the displayed data based on three primary criteria:
- Meter ID: Users can select a specific meter or view data for all meters.
- Event Type: Filters include tampering and various event flags (such as reverse current, magnetic field, cover open, neutral missing, voltage anomaly, and others).
- Date: Users can filter records by a specific date, enabling targeted analysis of historical events.
- These filters are implemented using HTML <select> and <input type="date"> elements, and the results are dynamically updated upon form submission, ensuring that only relevant data is displayed in the table.

8.3 Tabular Data Display and Tampered Row Highlighting

- Rows corresponding to tampered meters are visually distinguished using a unique background color (e.g., light red), making them immediately recognizable.

- The "Tampered" status is further emphasized with a colored badge within the table cell, ensuring that critical events stand out for rapid identification and response.
- The table is fully responsive, adapting to different screen sizes, and includes hover effects for improved readability and user experience.

8.4 Analysis Modal with Pie Charts

- An “Analysis” button opens a modal dashboard featuring multiple interactive pie charts that summarize key aspects of the filtered data:
- Tampered vs Not Tampered: Visualizes the proportion of tampered meters compared to those operating normally.
- Event Flags Causing Tampering: Breaks down the distribution of different event types (e.g., reverse current, magnetic field) among tampered meters.
- Categorical Distributions: Additional pie charts illustrate categorical variables such as meter location type, consumer type, bill payment history, and season, providing deeper insights into patterns associated with tampering events.
- These charts are dynamically generated using Chart.js, and their data updates in real time according to the current filter selection, enabling users to perform targeted exploratory analysis and quickly identify trends or anomalies.

8.5 External Dashboard Redirect

- When clicked the “External Dashboard” button, redirects users to an external analytics dashboard (e.g., a Streamlit application) for advanced data visualization, predictive analytics, or additional reporting features not available within the main interface.
- The button is visually distinct, featuring an icon and hover effects, ensuring it is easily accessible and recognizable for users seeking deeper analysis or alternative visual perspectives.

8.6 Additional Features

- User Experience: The page layout is clean and modern, with intuitive controls and clear feedback for filter actions, analysis modal toggling, and navigation.
- Security: All user-facing data is sanitized before display to prevent cross-site scripting (XSS) attacks, and only authenticated users can access the page.
- Reset and Navigation: Users can reset filters with a single click, returning the table to its default state, and navigate seamlessly between the data table, analysis modal, and external dashboard.

8.7 Data Table and Analysis Database Structure

The core of the analytical and reporting features is built upon the tampering database, specifically the datasheet_1 table. This table is designed to store high-frequency, granular event data for 10 distinct meters, with records captured at an hourly interval over the span of one year. This structure enables robust filtering, historical analysis, and visualization of meter activity, tampering events, and operational anomalies1.

Table Structure and Key Fields

- The datasheet_1 table is engineered to support comprehensive event logging and advanced analytics. Its main fields include:
- meter_id: Unique identifier for each of the 10 meters, enabling device-level filtering and aggregation.
- timestamp: Records the exact date and hour of each event, supporting time-series analysis and trend detection across a full year.
- consumption_kWh: Hourly energy consumption reading for each meter.
- load_variance: Measures the fluctuation in load, useful for anomaly and tampering detection.
- avg_daily_consumption: Rolling average of daily consumption, providing context for each hourly reading.

- peak_usage_shift: Categorical indicator showing when peak usage occurs (e.g., morning, afternoon, night).
- load_factor: Ratio indicating the efficiency of energy usage.
- reverse_current, magnetic_field_detected, cover_open_event, neutral_missing_event, voltage_anomaly, communication_loss: Event flags (binary/integer) that log the occurrence of specific tampering or fault events.
- meter_location_type, consumer_type, bill_payment_history, previous_tampering, day_of_week, season: Categorical fields supporting advanced filtering, segmentation, and pie chart analysis.
- tampering_label: Indicates tampering status (1 = tampered, 0 = normal), used for row highlighting and event analysis.
- This schema ensures that every meter's operational and event history is captured with hourly granularity, supporting both real-time monitoring and retrospective analytics1.

Data Usage in Reporting and Analysis

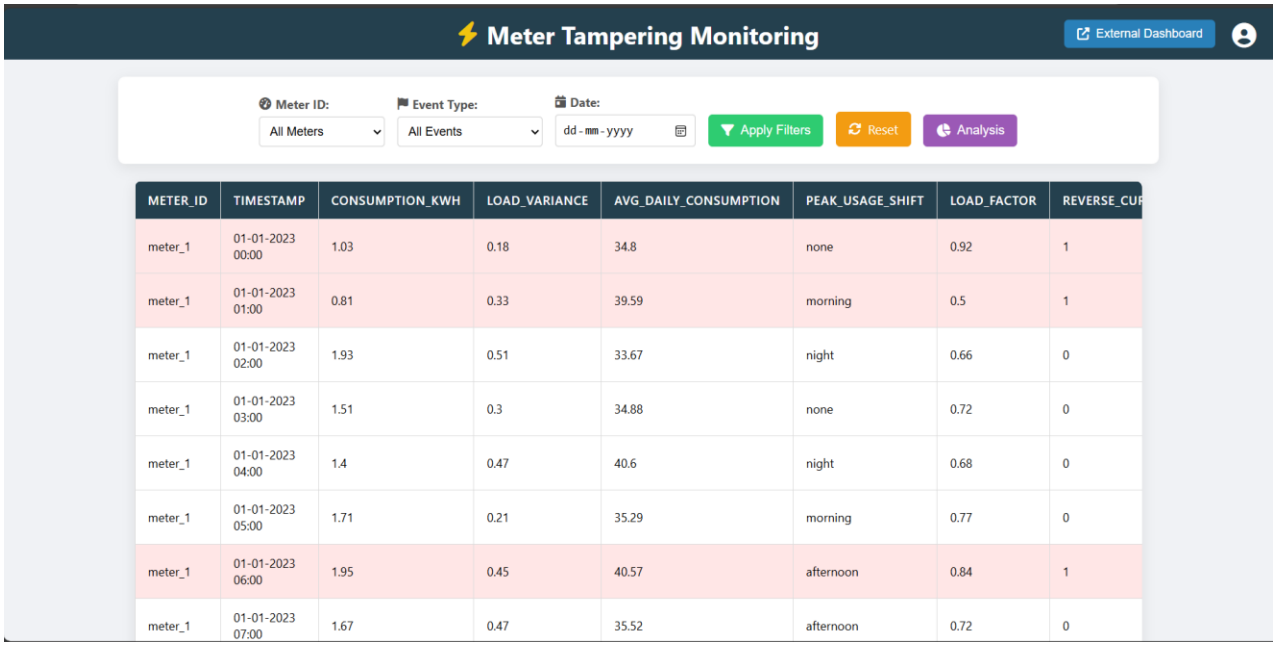
- Comprehensive Coverage: With 10 meters and hourly records for an entire year, the table contains approximately 87,600 entries (10 meters × 24 hours × 365 days), enabling statistically robust analysis and event detection1.
- Advanced Filtering: The table supports multi-criteria filtering by meter ID, event type, and date, allowing users to quickly isolate relevant records for investigation or reporting.
- Table Visualization: The web interface presents this data in a scrollable, responsive table. Tampered records are visually highlighted based on the tampering_label field, ensuring critical events are immediately visible.
- Pie Chart Analysis: Aggregated data from this table powers the analysis modal's pie charts, which summarize tampering rates, event flag distributions, and categorical breakdowns (e.g., by location type, consumer type, payment history, and season).
- Export and Integration: The structure supports easy export for further analysis or integration with external dashboards and reporting tools.

8.8 Data Table and Analysis Interface

meter_id	timestamp	consumption_kWh	load_variance	avg_daily_consumption	peak_usage_shift	load_factor	reverse_current	magnetic_field_detected	cover_open_event
meter_1	01-01-2023 00:00	1.03	0.18	34.8	none	0.92	1	0	0
meter_1	01-01-2023 01:00	0.81	0.33	39.59	morning	0.5	1	0	0
meter_1	01-01-2023 02:00	1.93	0.51	33.67	night	0.66	0	0	0
meter_1	01-01-2023 03:00	1.51	0.3	34.88	none	0.72	0	0	0
meter_1	01-01-2023 04:00	1.4	0.47	40.6	night	0.68	0	0	0
meter_1	01-01-2023 05:00	1.71	0.21	35.29	morning	0.77	0	0	0
meter_1	01-01-2023 06:00	1.95	0.45	40.57	afternoon	0.84	1	0	0
meter_1	01-01-2023 07:00	1.67	0.47	35.52	afternoon	0.72	0	0	0
meter_1	01-01-2023 08:00	1.95	0.38	37.89	afternoon	0.76	0	0	0
meter_1	01-01-2023 09:00	1.15	0.41	35.56	night	0.57	0	0	0
meter_1	01-01-2023 10:00	1.89	0.5	45.94	afternoon	0.77	0	0	0
meter_1	01-01-2023 11:00	1.85	0.26	33.28	none	0.59	1	0	0
meter_1	01-01-2023 12:00	1.79	0.41	42.84	afternoon	0.8	0	0	0

neutral_missing_event	voltage_anomaly	communication_loss	meter_location_type	consumer_type	bill_payment_history	previous_tampering	day_of_week	season	tampering_label
0	0	0	rural	residential	delinquent	0	Sunday	summer	1
0	0	0	indoor	commercial	delinquent	0	Sunday	winter	1
0	0	0	rural	residential	delinquent	0	Sunday	winter	0
0	0	0	outdoor	commercial	delinquent	0	Sunday	summer	0
0	0	0	indoor	residential	delinquent	0	Sunday	monsoon	0
0	0	0	urban	commercial	regular	0	Sunday	monsoon	0
0	0	0	urban	industrial	regular	0	Sunday	monsoon	1
0	0	0	indoor	commercial	regular	0	Sunday	monsoon	0
0	0	0	rural	commercial	delinquent	0	Sunday	summer	0
0	0	0	outdoor	industrial	delinquent	0	Sunday	winter	0
0	0	0	rural	industrial	irregular	0	Sunday	monsoon	0
0	0	0	rural	commercial	regular	0	Sunday	winter	1
0	0	0	urban	commercial	delinquent	0	Sunday	winter	0

Fig. 8.1



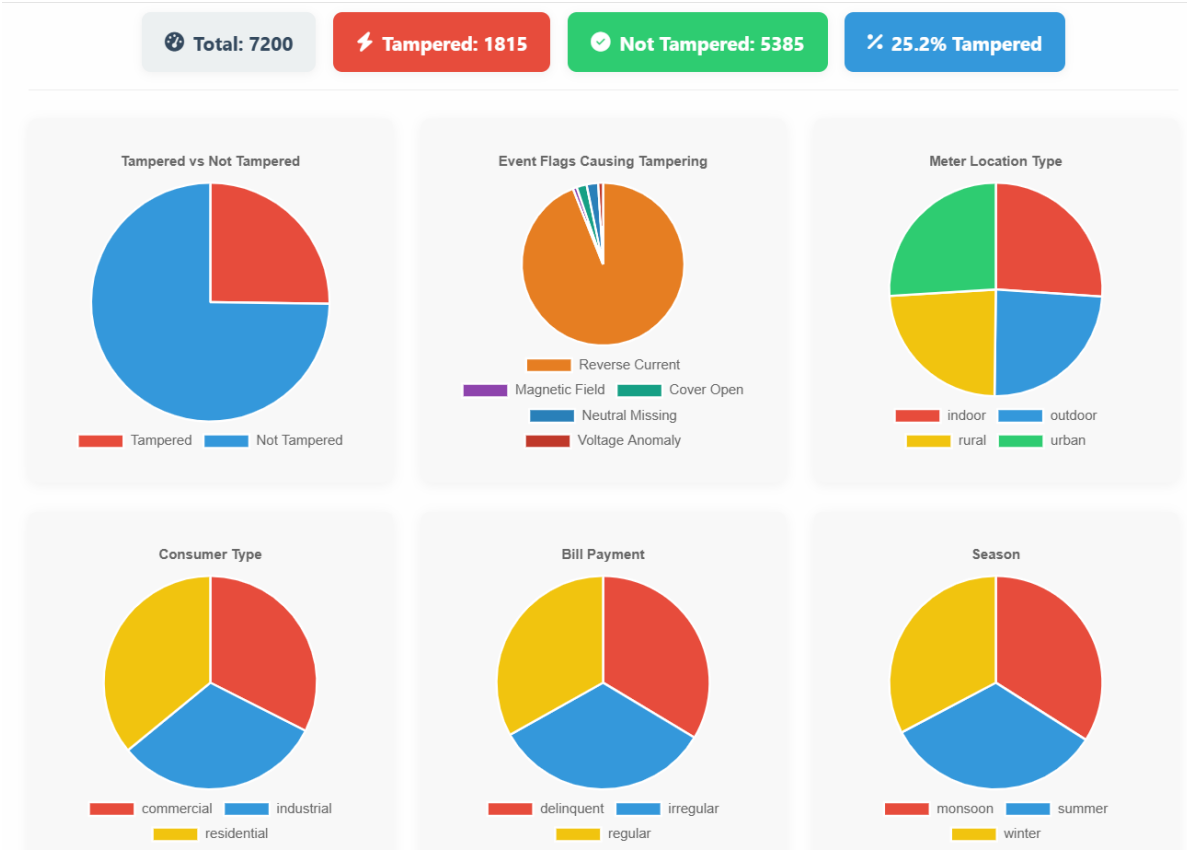

Total: 7200
Tampered: 1815
Not Tampered: 5385
25.2% Tampered
Tampered vs Not Tampered
Tampered
Not Tampered
Event Flags Causing Tampering
Reverse Current
Magnetic Field
Cover Open
Neutral Missing
Voltage Anomaly
Meter Location Type
indoor
outdoor
rural
urban
Consumer Type
commercial
industrial
residential
Bill Payment
delinquent
irregular
regular
Season
monsoon
summer
winter

Fig. 8.2

CHAPTER 9

STREAMLIT DASHBOARD DEPLOYMENT AND HOSTING

This section describes the process of deploying the interactive meter tampering dashboard as a web application, making it accessible to users through a streamlined web interface. The code implementation is detailed in the report in the section 5; here, the focus is on the deployment workflow and integration steps.

9.1 Preparing the GitHub Repository

- **Project Organization:**
    - All Streamlit dashboard code and dependencies were organized in a dedicated project folder.
    - A requirements.txt file was included to specify all necessary Python packages, such as streamlit, pandas, plotly, and scikit-learn.
    - The project folder was initialized as a Git repository using standard Git commands (git init, git add , git commit -m "Initial commit").
    - The repository was then pushed to GitHub, creating a public repository that hosts all app files and dependencies.

9.2 Deploying and Hosting the Streamlit App as a Website

- **Deployment Platform:**
    - The dashboard was deployed using **Streamlit Community Cloud**, a free platform for hosting Streamlit applications directly from GitHub repositories.
- **Deployment Steps:**

1. **Account Setup and Repository Connection:**
    - An account was created on Streamlit Community Cloud.

- The GitHub account was linked to Streamlit, authorizing access to the relevant repository.

2. **App Deployment:**

    - The “New app” button on the Streamlit Cloud dashboard was used to select the GitHub repository, specify the branch (main), and indicate the main app file (dashboard.py).
    - Upon deployment, Streamlit Cloud automatically installed dependencies from requirements.txt and built the app environment.

3. **URL Generation and Redirection:**

    - After deployment, Streamlit provided a unique URL for the live dashboard (https://predictioneb.streamlit.app/).
    - This URL was linked in the main project report page, allowing users to be redirected seamlessly to the live dashboard for interactive exploration.

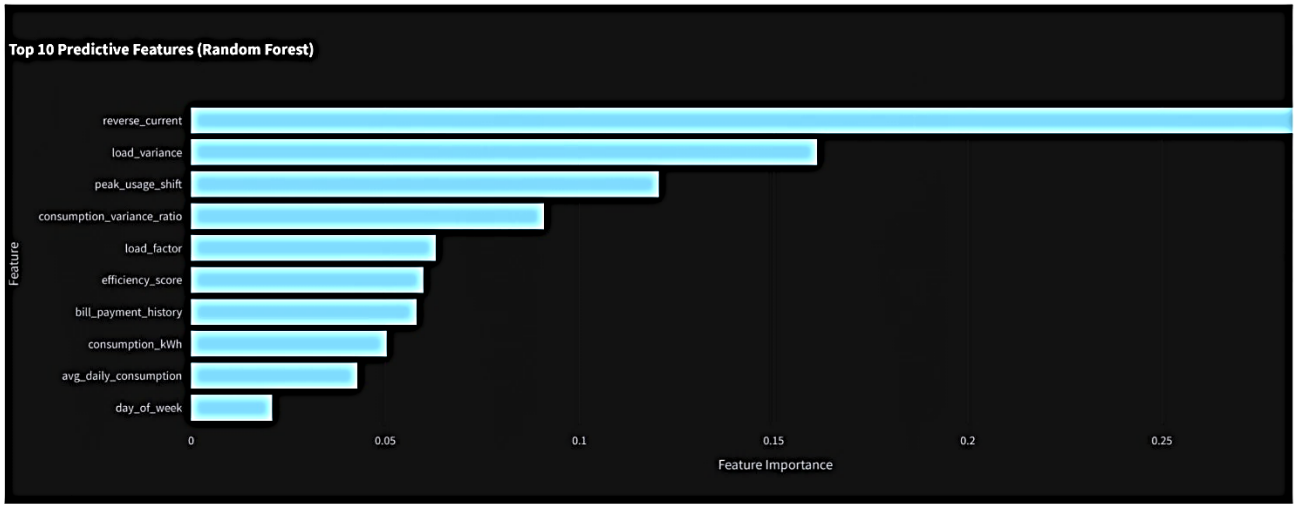
- **Maintenance and Updates:**
    - Any changes or improvements to the dashboard code were committed and pushed to the GitHub repository.
    - Changes can be made at any time to the dashboard code which can be made to reflect in the website by refreshing the app.
    - Streamlit Cloud automatically detected updates and rebuilt the app, ensuring the hosted site always reflected the latest version.

By leveraging GitHub for code management and Streamlit Community Cloud for hosting, the dashboard became accessible as a secure, interactive web application. This workflow enables real-time data visualization and tampering prediction.

Meter Tampering Forecast Dashboard (2024)

This dashboard forecasts 2024 meter consumption using ARIMA and predicts tampering alerts using a trained classifier.

Top Predictive Features



2024 Forecasted Consumption Example

	meter_id	timestamp	consumption_kWh	load_variance	avg_daily_consumption	peak_usage_shift	load_factor	reverse_current	magnetic_field_detected	cover_open_event	neutral_
0	meter_1	2024-01-31 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
1	meter_1	2024-02-01 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
2	meter_1	2024-02-02 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
3	meter_1	2024-02-03 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
4	meter_1	2024-02-04 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
5	meter_1	2024-02-05 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
6	meter_1	2024-02-06 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
7	meter_1	2024-02-07 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
8	meter_1	2024-02-08 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	
9	meter_1	2024-02-09 00:00:00	1.17	0.36	40.79	1	0.92	1	0	0	

Multi-Dimensional Tampering Analysis

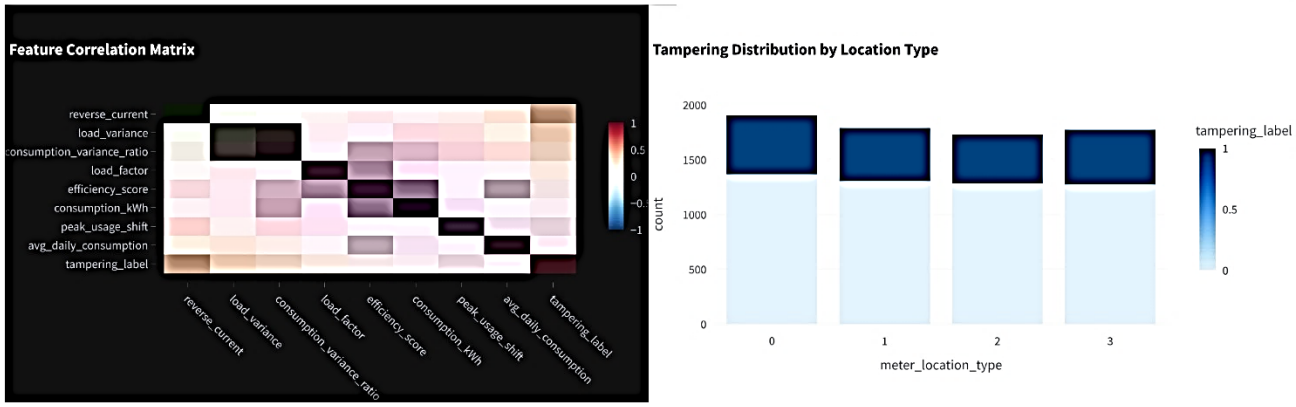
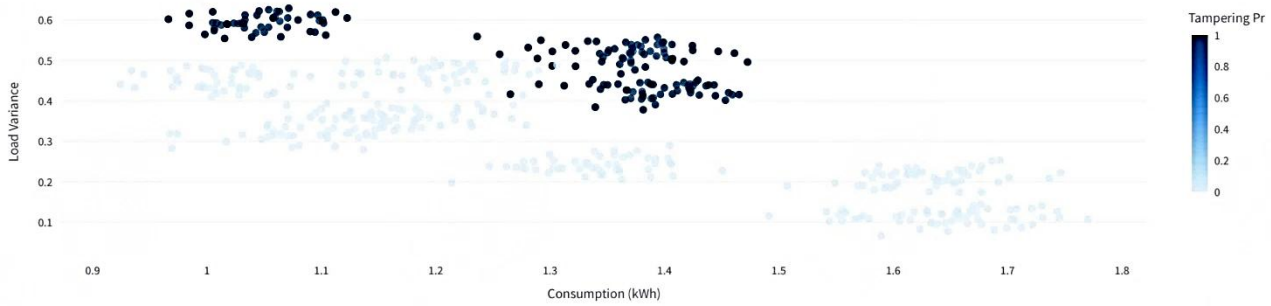


Fig. 9.1

Consumption Patterns vs Tampering (2024)

Tampering Pattern by Load Variance (2024)



Predicted Tampering Alerts for 2024

	meter_id	timestamp	consumption_kWh	tamper_prediction
732	meter_3	2024-01-31 00:00:00	1.4	
733	meter_3	2024-02-01 00:00:00	1.4	
734	meter_3	2024-02-02 00:00:00	1.4	
735	meter_3	2024-02-03 00:00:00	1.4	
736	meter_3	2024-02-04 00:00:00	1.4	
737	meter_3	2024-02-05 00:00:00	1.4	
738	meter_3	2024-02-06 00:00:00	1.4	
739	meter_3	2024-02-07 00:00:00	1.4	
740	meter_3	2024-02-08 00:00:00	1.4	
741	meter_3	2024-02-09 00:00:00	1.4	

Download 2024 Tampering Alerts

Fig. 9.2

CHAPTER 10

PERFORMANCE EVALUATION METRICS

To evaluate the effectiveness of our tampering detection and forecasting models, we used key metrics that reflect both classification accuracy (for Random Forest) and time-series forecasting accuracy (for ARIMA).

10.1 Metrics for Tampering Detection (Random Forest)

- **Accuracy**: Overall correctness of predictions (normal vs tampered).
- **Precision**: How many predicted tampered meters were actually tampered.
- **Recall**: How many actual tampering cases the model correctly identified (important for catching theft).
- **F1-Score**: Balances precision and recall—critical since tampering is rare and imbalanced.

In our project, high recall (~85%) was prioritized to avoid missing actual tampering events.

10.2 Metrics for Tampering Forecasting (ARIMA)

- **Mean Absolute Error (MAE)**: Measures average error in predicted tampering probability.
- **MAPE (Mean Absolute Percentage Error)**: Shows how accurate our tampering forecast is in percentage terms.

Our ARIMA model achieved MAPE < 10%, indicating good forecast accuracy over 2024 based on 2023 data.

CHAPTER 11

CHALLENGES AND FUTURE ENHANCEMENTS

Despite the successful design and implementation of a multi-stage tampering detection and prediction system, several challenges and limitations were encountered throughout the project. These constraints are important to acknowledge as they impact model accuracy, scalability, and real-world deployment feasibility.

11.1 Data Noise and Missing Values

- **Challenge:** The raw data imported from AMR systems (simulated through Excel in this case) often includes missing entries or sudden spikes due to communication loss, power outages, or logging errors.
- **Impact:** These inconsistencies affect time-series continuity and model reliability. While basic interpolation techniques were used to handle small gaps, larger gaps reduce the accuracy of both tampering detection and future forecasting.

11.2 Uncertainty in Tampering Label Assignment

- **Challenge:** Tampering labels were derived indirectly through simulation using threshold-based logic, which may not always reflect actual field conditions. For example, a drop in current might also be caused by a genuine reduction in load.
- **Impact:** The absence of “ground truth” labels limits the supervised learning model's ability to learn true tampering behaviour, and increases the chance of false positives or negatives.

11.3 Imbalanced Dataset

- **Challenge:** In real-world and simulated data, tampering events are relatively rare compared to normal readings, resulting in class imbalance.

- **Impact:** Machine learning models tend to be biased towards the majority class (normal readings), making it harder to detect rare but important tampering instances.

11.4 Generalization Limitations of Machine Learning Models

- **Challenge:** The Random Forest classifier and ARIMA forecasting model were trained on historical data for specific meters and tampering scenarios.
- **Impact:** These models may not generalize well to unseen regions, meters, or consumers whose behaviour patterns differ significantly. Additional training with diverse, real-time data is needed to improve adaptability..

These limitations are typical of early-stage proof-of-concept projects. Despite them, the system demonstrates a strong foundation for multi-parameter tampering analysis and predictive modeling. Addressing these challenges in future work will significantly improve the system's accuracy, and real-world applicability.

11.5 Future Enhancements

While the current system provides a solid foundation for simulating and predicting electricity meter tampering using machine learning and web-based visualization, several opportunities exist to enhance accuracy, and real-time responsiveness. The following future enhancements are proposed to extend the system's capability into a fully deployable smart-grid solution.

Real-Time Data Ingestion with Kafka or MQTT

- **Proposed Enhancement:** Integrate real-time stream processing using messaging platforms like **Apache Kafka** or **MQTT** to collect live meter data from deployed AMR/AMI devices.
- **Expected Benefit:** Enables continuous monitoring, immediate anomaly detection, and predictive insights without depending on static Excel or CSV files. It also allows the forecasting system to update models dynamically as new data arrives.

Adoption of Deep Learning Models (LSTM/GRU)

- **Proposed Enhancement:** Replace or augment ARIMA and Random Forest models with **Long Short-Term Memory (LSTM)** or **Gated Recurrent Unit (GRU)** models for sequential data prediction.
- **Expected Benefit:** LSTM-based models can better capture long-term dependencies and time-based patterns in consumption behaviour, which are essential for identifying subtle or periodic tampering trends.

Smart Grid Integration via Open APIs

- **Proposed Enhancement:** Develop **REST APIs** or **WebSocket connections** that link the dashboard with smart grid control units or IoT platforms.
- **Expected Benefit:** Allows automated responses such as isolating tampered meters, flagging suspicious usage to authorities, or adjusting load balance in real time.

Alerting and Notification System

- **Proposed Enhancement:** Integrate **email, SMS, or push notification services** (e.g., Twilio, SendGrid) to alert grid operators or customers when tampering is predicted.
- **Expected Benefit:** Shifts the model from passive detection to **active prevention**, enabling timely interventions that reduce energy loss and damage.

Mobile App or Progressive Web App (PWA) Interface

- **Proposed Enhancement:** Build a mobile version of the dashboard using **Streamlit Mobile**, **React Native**, or a **PWA** wrapper.
- **Expected Benefit:** Field officers or technicians can receive tampering alerts and reports directly on smartphones or tablets, improving response efficiency.

Model Retraining and Feedback Loop

- **Proposed Enhancement:** Enable feedback from field validations (e.g., confirmed tampering cases) to be used to **continuously retrain the models**.
- **Expected Benefit:** Ensures models evolve over time, adapting to new tampering techniques or changes in consumer behaviour.

CHAPTER 12

CONCLUSION

The growing challenge of energy theft through smart meter tampering necessitates the development of more intelligent and proactive solutions. This project presents a comprehensive framework that not only detects tampering based on current, voltage, and multiple event flags, but also predicts future tampering behavior using statistical and machine learning models.

Through the use of SIMULINK and SCILAB, various real-world tampering scenarios - such as sudden current drops, periodic tampering, and random consumption anomalies were effectively simulated. These simulations formed the foundation for data-driven tampering detection, enabling the identification of unusual patterns in power usage and meter activity.

The forecasting models, developed using ARIMA and Random Forest, leveraged these patterns to anticipate future tampering events. These predictions can empower utility providers to perform targeted inspections, reduce technical losses, and optimize maintenance schedules.

To make the system accessible and user-friendly, a complete web-based dashboard was developed using Streamlit, along with secure login, an interactive map of faulty meters, and real-time visualizations. This seamless integration of simulation, detection, prediction, and deployment showcases a full-stack solution that bridges the gap between raw meter data and actionable insights.

While the system has demonstrated promising results, it currently operates on simulated and batch-uploaded data. Future work can extend this framework with real-time AMR streams, IoT device integration, advanced deep learning models, and automated alerting mechanisms to move closer to a fully autonomous tampering prevention system.

Ultimately, this project illustrates how combining simulation, AI, and web technologies can enhance the intelligence, accuracy, and scalability of modern energy monitoring systems-enabling smarter, safer, and more efficient power distribution.

REFERENCES

1) “Meter tampering — the major cause of non-technical losses”

https://www.smart-energy.com/industry-sectors/smart-meters/meter-tampering-the-major-cause-of-non-technical-losses/

2) "Pros and Cons of Tamper Detection with AMR" – Smart Energy International

https://www.smart-energy.com/wp-content/uploads/Ed%20Malemezian.pdf?

3) Modern computerized meters and AMR setups use multiple sensors to detect tampering, including:

- Cover access sensors
- Magnetic interference detection
- Phase/neutral reversal or inversion
- Meter bypass alarms when neutral current increases unexpectedly

Prevent tampering in energy meters - EDN

Prevent Tampering in Energy Meters - EE Times

SSZT372 Technical article | TI.com

NOTE: akhshaiyas/TampriX – GitHub repository with all the code files.


WAN Comm. : Cellular - 4G/2G
(800/900/1800/1900 MHz)
IMEI No: 864180054727974
VISIONTEK
Smart Meter for General and Solar Application
SM-38
ac STATIC TRANSFORMER OPERATED SMART ENERGY METER
IS:16444 (Pt-2)
CM/L-6300074203
Principal Units : kWh, kVArh, kVAh, kW, kVA
4000 Imp/kWh
4000 imp/kVArh(or)kVAh
Unique identification number
TGCCTLWXXXXXXXX
S.No.
Month/Year
99000762 02/24
Make & Model
LW_SM-38
Optical Port
Mfr.: LINKWELL TELESYSTEMS PVT.LTD., HYDERABAD, INDIA.
AC STATIC 3 PHASE 4 WIRE LTCT OPERATED SMART ENERGY METER
SL.No: 04007331
Tamil Nadu Generation and Distribution Corporation Limited
Property of
"TANGEDCO"
Mfr. : Inesh Smart Energy (P) Ltd. Chennai, India.
Powered by Exhilar Innovative Solutions Pvt. Ltd.
WAN Communication - 4G
cellular(900| 1800 | 2100 | 2655 MHz)
SMART METER


CM/L-6300074203
00000
TAMPER
Principal Units : kWh, kVArh, kVAh, kW, kVA
WAN Comm. : Cellular - 4G/2G
(800/900/1800/1900 MHz)
IMEI No: 864180054727974
SIM
NW
RxD
TxD
Mfr.: LINKWELL TELESYSTEMS PVT.LTD., HYDERABAD, INDIA.


VISIONTEK
Smart Meter for General and Solar Application
SM-38
ac STATIC TRANSFORMER OPERATED SMART ENERGY METER
IS:16444 (Pt-2)
CM/L-6300074203
TAMPER
3Ph-4W 3x240V -/5A
50Hz Cl.0.5S Ref. 27°C
IS15959 (Pt-3) Cat-D3
Principal Units : kWh, kVArh, kVAh, kW, kVA
4000
4000 Imp/
PCFR
STABILISED